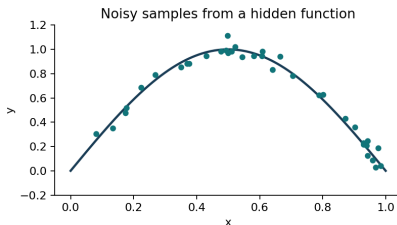


The supervised learning task

- We observe labelled examples (x_i, y_i) and want a predictor u_θ that maps a new input x to a plausible label.
- Here we fit the model to input–output pairs. We will add PDE constraints in Chapter 3.
- How well does u_θ predict data that were not used to fit θ ?



Dataset notation and tensor shapes

- Store the inputs as rows and the labels as a column:

$$X = \begin{bmatrix} x_1^\top \\ \vdots \\ x_N^\top \end{bmatrix} \in \mathbb{R}^{N \times d}, \quad Y = \begin{bmatrix} y_1 \\ \vdots \\ y_N \end{bmatrix} \in \mathbb{R}^{N \times 1}.$$

- For scalar regression, the network output should have the same shape:

$$\hat{Y}_\theta = u_\theta(X) = \begin{bmatrix} u_\theta(x_1) \\ \vdots \\ u_\theta(x_N) \end{bmatrix} \in \mathbb{R}^{N \times 1}.$$

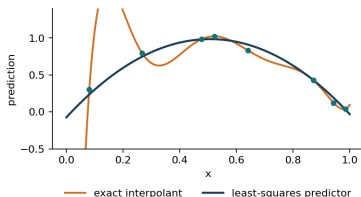
- Check tensor shapes to catch silent broadcasting bugs, such as $(N, 1) - (N,)$ becoming an (N, N) matrix of residuals.

Interpolation as a starting point

- Classical interpolation asks for exact agreement at prescribed points:

$$p(x_i) = y_i, \quad i = 1, \dots, N.$$

- Polynomial interpolation uses one global polynomial; piecewise-linear interpolation connects neighbours; cubic splines use smooth piecewise cubics.
- For noisy data, we usually fit an average trend rather than require exact agreement at every point.



Supervised regression is not interpolation

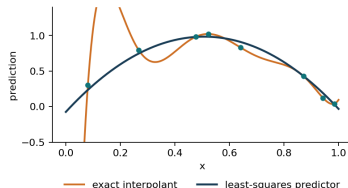
- Interpolation imposes constraints:

$$u(x_i) = y_i \quad \text{for every observed point.}$$

- Regression minimises an average discrepancy:

$$\min_{\theta} \frac{1}{N} \sum_{i=1}^N (u_{\theta}(x_i) - y_i)^2.$$

- With noisy labels, exact agreement can be undesirable. A predictor may generalise better by not passing through every sample.



The data-generating model

- ▶ A useful statistical model is

$$Y = f_{\star}(X) + \varepsilon, \quad \mathbb{E}[\varepsilon \mid X] = 0.$$

- ▶ The unknown function $f_{\star}$ is the signal; ε represents measurement noise or unresolved variability.
- ▶ Training data are finite samples:

$$\mathcal{D}_N = \{(x_i, y_i)\}_{i=1}^N \sim \rho^{\otimes N}.$$

- ▶ Generalisation means predicting new samples from the same distribution accurately.

Population risk: the target quantity

- The ideal squared-loss risk is

$$R(\theta) = \mathbb{E}_{(X,Y) \sim \rho} \left[(u_{\theta}(X) - Y)^2 \right].$$

- If $Y = f_{\star}(X) + \varepsilon$ and $\mathbb{E}[\varepsilon \mid X] = 0$, then

$$R(\theta) = \|u_{\theta} - f_{\star}\|_{L^2(\rho_X)}^2 + \mathbb{E}[\varepsilon^2].$$

- The noise term is irreducible for this input. No deterministic model can remove it.

Empirical risk: the computable quantity

- ▶ The population expectation is replaced by a finite average:

$$R_N(\theta) = \frac{1}{N} \sum_{i=1}^N (u_{\theta}(x_i) - y_i)^2.$$

- ▶ Training means approximately solving

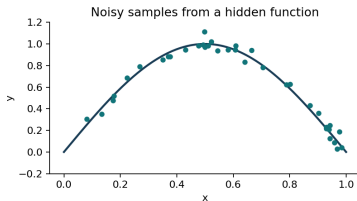
$$\hat{\theta} \approx \arg \min_{\theta \in \Theta} R_N(\theta).$$

- ▶ Training is approximate: we stop after finitely many updates and may not find a global minimum. The finite-data loss also differs from population risk.

Generating a toy dataset in code

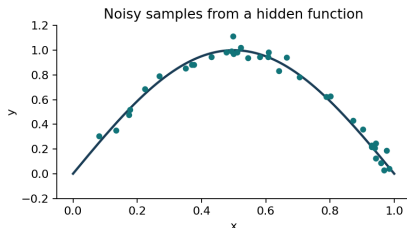
- ▶ A simple regression test case is noisy data from a smooth target.
- ▶ Fix the data and random seed before comparing models.

```
rng = np.random.default_rng(4)
x = np.sort(rng.uniform(0, 1, 64))
   [:, None]
y_clean = np.sin(np.pi * x)
y = y_clean + 0.05 * rng.normal(size
    =x.shape)
```



What the plot tells us before training

- ▶ The samples suggest a smooth one-dimensional pattern, but the labels are not exactly on the hidden curve.
- ▶ A model that follows every noisy point may have low training error and higher prediction error.
- ▶ A model that is too rigid may miss the curvature even if it is stable.



Start with an affine baseline

- ▶ The simplest model is

$$u_{w,b}(x) = wx + b.$$

- ▶ The residuals are

$$r_i = wx_i + b - y_i.$$

- ▶ The training objective is a convex quadratic:

$$L(w, b) = \frac{1}{N} \sum_{i=1}^N r_i^2.$$

- ▶ We can solve this problem directly, then see how much error comes from fitting a straight line.

Linear regression in matrix form

- Introduce the design matrix

$$A = \begin{bmatrix} x_1 & 1 \\ \vdots & \vdots \\ x_N & 1 \end{bmatrix}, \quad \beta = \begin{bmatrix} w \\ b \end{bmatrix}.$$

- The fitted values are $A\beta$ and the empirical loss is

$$L(\beta) = \frac{1}{N} \|A\beta - Y\|_2^2.$$

- The gradient is

$$\nabla_{\beta} L(\beta) = \frac{2}{N} A^{\top} (A\beta - Y).$$

The normal equations

- ▶ Setting the gradient to zero gives

$$A^{\top} A \hat{\beta} = A^{\top} Y.$$

- ▶ If $A^{\top} A$ is nonsingular, then

$$\hat{\beta} = (A^{\top} A)^{-1} A^{\top} Y.$$

- ▶ In practice, a least-squares solver is more stable than explicitly forming the inverse.
- ▶ Here we can find a global minimiser by solving a linear system.

The baseline in NumPy

- ▶ The two-column matrix stores the slope feature and the intercept feature.

```
A = np.column_stack([train_x[:, 0], np.ones(len(train_x))])
beta, *_ = np.linalg.lstsq(A, train_y[:, 0], rcond=None)
w, b = beta
prediction = w * test_x[:, 0] + b
mse = np.mean((prediction - test_y[:, 0])**2)
```

- ▶ The optimisation error is essentially zero; any remaining error is due to model rigidity and finite data.

What the affine baseline cannot express

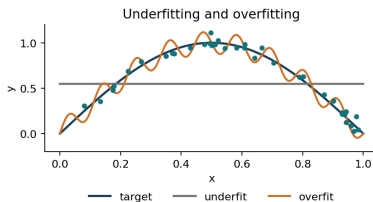
- ▶ A straight line has no curvature:

$$\frac{d^2}{dx^2}(wx + b) = 0.$$

- ▶ The hidden target $\sin(\pi x)$ has curvature:

$$\frac{d^2}{dx^2} \sin(\pi x) = -\pi^2 \sin(\pi x).$$

- ▶ Even the best straight line will miss this curvature.



A neural network as nested functions

- ▶ A two-hidden-layer scalar network can be written as

$$a^{(1)} = \sigma(W_1 x + b_1), \quad a^{(2)} = \sigma(W_2 a^{(1)} + b_2),$$

$$u_{\theta}(x) = W_3 a^{(2)} + b_3.$$

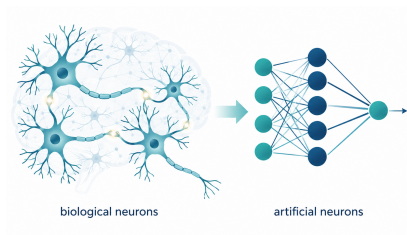
- ▶ The parameter vector is

$$\theta = (W_1, b_1, W_2, b_2, W_3, b_3).$$

- ▶ Training chooses the weights and biases from data; the architecture specifies the allowed compositions.

Why call it a neural network?

- ▶ The name comes from an analogy with biological neural systems: many simple units send signals along connections.
- ▶ In the artificial model, the unit is much simpler: it computes a weighted sum, adds a bias, and applies a scalar nonlinearity.
- ▶ The biological analogy explains the name. Mathematically, we compose parametrised functions.



From a biological neuron to an artificial one

- ▶ A biological neuron receives many inputs through dendrites and sends an output signal along an axon.
- ▶ An artificial neuron receives numbers $x_1, \dots, x_d$ and computes one number:

$$z = w \cdot x + b = \sum_{j=1}^d w_j x_j + b, \quad a = \sigma(z).$$

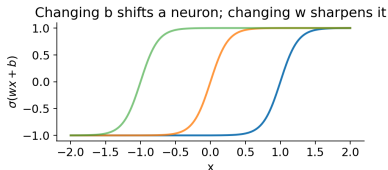
- ▶ Weights play the role of connection strengths. The activation σ is a simple firing rule, not a realistic model of cell biology.
- ▶ A layer repeats this computation in parallel.

One artificial neuron

- ▶ A single neuron has parameters (w, b) and output

$$x \mapsto \sigma(w \cdot x + b).$$

- ▶ The hyperplane $w \cdot x + b = 0$ is the transition region of the neuron.
- ▶ Larger $\|w\|$ makes the transition sharper; changing b shifts it.



A layer as parallel neurons

- ▶ A layer with m neurons computes

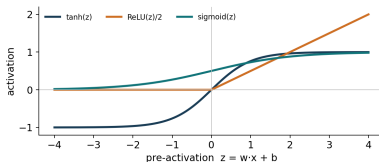
$$a(x) = \sigma(Wx + b) \in \mathbb{R}^m,$$

where $W \in \mathbb{R}^{m \times d}$ and $b \in \mathbb{R}^m$.

- ▶ Componentwise, this is

$$a_j(x) = \sigma(w_j \cdot x + b_j), \quad j = 1, \dots, m.$$

- ▶ The layer turns the input into m learned features. The next layer combines those features.



A shallow network

- ▶ A one-hidden-layer network has the form

$$u_n(x) = c_0 + \sum_{j=1}^n c_j \sigma(w_j \cdot x + b_j).$$

- ▶ The inner parameters (w_j, b_j) choose the features. The outer coefficients c_j combine them.
- ▶ The approximation family is

$$\mathcal{M}_n(\sigma) = \left\{ c_0 + \sum_{j=1}^n c_j \sigma(w_j \cdot x + b_j) \right\}.$$

- ▶ Universal approximation asks whether $\bigcup_n \mathcal{M}_n(\sigma)$ is dense in a function space.

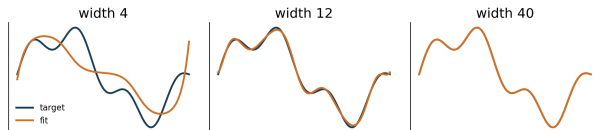
What universal approximation means

- ▶ A density statement has the form

$$\forall f \in C(K), \quad \forall \varepsilon > 0, \quad \exists n, \exists u_n \in \mathcal{M}_n(\sigma) : \|f - u_n\|_{\infty, K} < \varepsilon.$$

- ▶ This is an existence theorem. It says the family contains a good approximant.
- ▶ It does not say how large n must be, how large the weights are, or whether gradient-based training will find the approximant.

Same activation family, more hidden units



Cybenko's universal approximation theorem

Theorem (Cybenko, 1989)

Let $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ be continuous and sigmoidal, meaning

$$\lim_{t \rightarrow -\infty} \sigma(t) = 0, \quad \lim_{t \rightarrow +\infty} \sigma(t) = 1.$$

Then finite sums

$$\sum_{j=1}^n c_j \sigma(w_j \cdot x + b_j)$$

are dense in $C([0, 1]^d)$ in the uniform norm.

- The width may grow with the target function and the requested accuracy.

Cybenko proof sketch: separate by a functional

- ▶ Let S be the uniform closure of all finite sums of neurons.
- ▶ Suppose, for contradiction, that $S \neq C([0, 1]^d)$.
- ▶ By Hahn–Banach, there is a nonzero continuous linear functional L such that

$$L(g) = 0 \quad \text{for every } g \in S.$$

- ▶ By the Riesz representation theorem, there is a nonzero signed measure μ with

$$L(g) = \int_{[0,1]^d} g(x) d\mu(x).$$

Cybenko proof sketch: annihilating neurons

- ▶ Since every neuron lies in S , the measure satisfies

$$\int_{[0,1]^d} \sigma(w \cdot x + b) d\mu(x) = 0 \quad \text{for all } w, b.$$

- ▶ The sigmoidal property lets scaled neurons approximate half-space indicators:

$$\sigma(\lambda(w \cdot x + b)) \longrightarrow \mathbf{1}_{\{w \cdot x + b > 0\}} \quad \text{as } \lambda \rightarrow \infty.$$

- ▶ Passing to the limit shows that μ assigns zero mass to all half-spaces.

Cybenko proof sketch: the contradiction

- ▶ If a finite signed measure vanishes on all half-spaces, then it is the zero measure.
- ▶ One way to see the last step is through Fourier analysis: half-space information determines the Fourier transform of the measure.
- ▶ Thus $\mu = 0$, contradicting the nonzero measure supplied by Hahn–Banach.
- ▶ Therefore the closure S must be the whole space $C([0, 1]^d)$.

The Pinkus / LLPS sharpening

Theorem (Leshno–Lin–Pinkus–Schocken, 1993)

For continuous σ , the family of one-hidden-layer networks is dense in $C(K)$ for every compact $K \subset \mathbb{R}^d$ if and only if σ is not a polynomial.

- ▶ The activation can be any continuous non-polynomial function; it need not be sigmoidal.
- ▶ ReLU, tanh, sigmoid, softplus and many other non-polynomial activations satisfy the density condition.

Why polynomials are the obstruction

- ▶ If σ is a polynomial of degree m , then every neuron $\sigma(w \cdot x + b)$ is also a polynomial of degree at most m .
- ▶ Finite sums remain in the same finite-dimensional polynomial space.
- ▶ A finite-dimensional proper subspace of $C(K)$ is closed and cannot be dense.
- ▶ Thus a polynomial activation cannot be universal.

Pinkus proof sketch I: stay inside the span

- ▶ In one dimension, consider the dictionary functions $x \mapsto \sigma(wx + b)$.
- ▶ Difference quotients in the inner weight remain in the span before taking the limit:

$$\frac{\sigma((w + h)x + b) - \sigma(wx + b)}{h}.$$

- ▶ If σ is smooth, the limit gives

$$x \sigma'(wx + b) \in \overline{\text{span}\{\sigma(wx + b)\}}.$$

- ▶ Iterating the argument produces

$$x^k \sigma^{(k)}(wx + b) \quad \text{in the closure.}$$

Pinkus proof sketch II: recover monomials

- Set $w = 0$ in the previous expression:

$$x^k \sigma^{(k)}(b) \in \overline{\text{span}}.$$

- If σ is not a polynomial, then for every k there is some b_k with

$$\sigma^{(k)}(b_k) \neq 0.$$

- Hence each monomial x^k lies in the closure of the network span.
- Since polynomials are dense in $C(K)$ by Weierstrass, the network span is dense.

Pinkus proof sketch III: remove smoothness

- ▶ The previous argument was written for smooth σ to show the mechanism.
- ▶ For merely continuous activations, convolve with a smooth mollifier:

$$\sigma_\delta = \sigma * \varphi_\delta.$$

- ▶ Translates of σ_δ are limits of finite sums of translates of σ , so the smoothed activation remains in the closure generated by σ .
- ▶ Apply the smooth argument to a non-polynomial mollification and then pass back to σ .

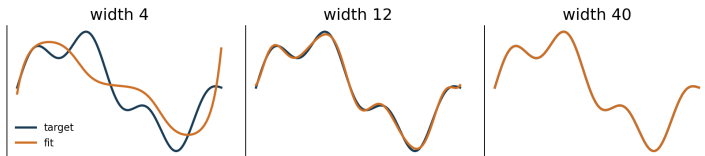
What remains after an approximation theorem

- ▶ We still need an algorithm to find the weights.
- ▶ We need a practical width for the requested accuracy.
- ▶ The weights may be large.
- ▶ Error on the training data can differ from population risk.
- ▶ Optimisation and generalisation need separate analysis.

Universal approximation in a toy plot

- ▶ Increasing the number of hidden units gives a larger function family.
- ▶ In this least-squares example, adding hidden units makes the fitted curve more flexible.
- ▶ The theorem explains why approximation is possible in principle; the experiment still depends on feature placement, conditioning, and optimisation.

Same activation family, more hidden units



From approximation to training

- ▶ The theorems justify using neural-network families as flexible approximators.
- ▶ Next we work through training and evaluation:

data $\rightarrow$ empirical loss $\rightarrow$ optimiser $\rightarrow$ validation/test error.

- ▶ There are three sources of error to consider: approximation, optimisation, and estimation.
- ▶ Universal approximation addresses only the first one.

One hidden layer is enough for density

- ▶ The universal approximation theorems already apply to shallow networks:

$$u_n(x) = c_0 + \sum_{j=1}^n c_j \sigma(w_j \cdot x + b_j).$$

- ▶ Depth is not needed to obtain density in $C(K)$.
- ▶ Depth can still matter for efficiency: some compositional functions may need fewer parameters when represented by deeper architectures.
- ▶ Density alone leaves open the required model size, training cost, and amount of data.

Why existence does not settle training

- The theorem states existence:

$$\inf_{u \in \mathcal{M}_n} \|f - u\| \rightarrow 0.$$

- Training solves a different finite-data problem:

$$\hat{\theta} \approx \arg \min_{\theta} \frac{1}{N} \sum_{i=1}^N (u_{\theta}(x_i) - y_i)^2.$$

- To find a useful fit from finite data, we also need optimisation, regularisation, and independent evaluation.

Shape bookkeeping for a width-32 MLP

- ▶ With batch size N and scalar input, the tensors have shapes

$$X : (N, 1), \quad A^{(1)} : (N, 32), \quad A^{(2)} : (N, 32), \quad U_{\theta} : (N, 1).$$

- ▶ The matrix dimensions are

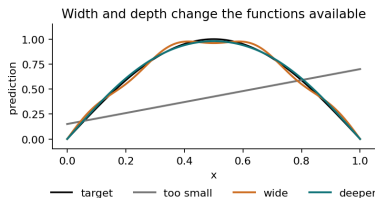
$$W_1 : (1, 32), \quad W_2 : (32, 32), \quad W_3 : (32, 1),$$

using the common row-sample convention.

- ▶ Check these dimensions before computing the loss.

Width and depth as modelling choices

- ▶ Width increases the number of hidden features available at a given layer.
- ▶ Depth increases the number of nonlinear compositions.
- ▶ More width or depth can reduce approximation error, but it can also increase estimation difficulty and optimisation sensitivity.
- ▶ Compare models using the same data splits and training budget, then check validation error.



Counting parameters

- ▶ For a network $1 \rightarrow 32 \rightarrow 32 \rightarrow 1$, the parameter count is

$$(1 \cdot 32 + 32) + (32 \cdot 32 + 32) + (32 \cdot 1 + 1).$$

- ▶ Therefore

$$64 + 1056 + 33 = 1153.$$

- ▶ Record the parameter count when comparing models; accuracy also depends on training and data.
- ▶ With only tens of training points, the model can be much larger than the dataset.

The PyTorch model

- ▶ The model definition states the architecture, activation, and numerical dtype.

```
model = torch.nn.Sequential(  
    torch.nn.Linear(1, 32, dtype=torch.float64),  
    torch.nn.Tanh(),  
    torch.nn.Linear(32, 32, dtype=torch.float64),  
    torch.nn.Tanh(),  
    torch.nn.Linear(32, 1, dtype=torch.float64),  
)  
assert sum(p.numel() for p in model.parameters()) == 1153
```

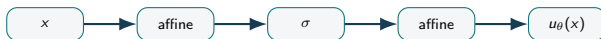
- ▶ The final layer is linear because this is scalar regression, not classification.

Why an activation is necessary

- ▶ A composition of affine maps is affine:

$$W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2).$$

- ▶ Without a nonlinear activation, depth would not create a nonlinear predictor.
- ▶ A nonlinear activation lets the network represent curved functions.

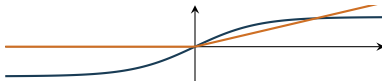


Smooth and piecewise-linear activations

- Two common choices are

$$\tanh z = \frac{e^z - e^{-z}}{e^z + e^{-z}}, \quad \text{ReLU}(z) = \max\{0, z\}.$$

- ReLU networks are piecewise linear and often train efficiently.
- Smooth activations such as tanh produce smooth predictors, which is useful when derivatives of the fitted function are inspected.



Forward pass for one input

- For a single scalar input x , the forward pass is the sequence

$$z^{(1)} = W_1x + b_1, \quad a^{(1)} = \sigma(z^{(1)}),$$

$$z^{(2)} = W_2a^{(1)} + b_2, \quad a^{(2)} = \sigma(z^{(2)}).$$

$$u_\theta(x) = W_3a^{(2)} + b_3.$$

- The intermediate activations are stored because the backward pass reuses them.

Forward pass for a batch

- ▶ A batch evaluates many inputs at once:

$$Z^{(1)} = XW_1 + \mathbf{1}b_1^\top, \quad A^{(1)} = \sigma(Z^{(1)}).$$

- ▶ The same weights are shared across all rows:

$$U_\theta(X) = A^{(2)}W_3 + \mathbf{1}b_3.$$

- ▶ Batching changes the computation schedule, not the mathematical function represented by the network.

The loss in PyTorch

- The empirical squared loss is a scalar average over samples:

$$L_{\text{data}}(\theta) = \frac{1}{N} \sum_{i=1}^N (u_{\theta}(x_i) - y_i)^2.$$

```
prediction = model(train_x)
if prediction.shape != train_y.shape:
    raise ValueError("prediction and target shapes must agree")
loss = ((prediction - train_y)**2).mean()
assert loss.ndim == 0
```

- Dividing by the parameter count would define a different objective and rescale the gradients.

Gradients for the affine baseline

- For $u_{w,b}(x) = wx + b$ and residual $r_i = wx_i + b - y_i$,

$$L(w, b) = \frac{1}{N} \sum_i r_i^2.$$

- The derivatives are

$$\frac{\partial L}{\partial w} = \frac{2}{N} \sum_i r_i x_i, \quad \frac{\partial L}{\partial b} = \frac{2}{N} \sum_i r_i.$$

- Gradient descent replaces the closed-form solve by repeated small parameter updates.

Gradient descent as repeated local improvement

- ▶ Given a differentiable objective $L(\theta)$, gradient descent uses

$$\theta_{k+1} = \theta_k - \eta \nabla L(\theta_k).$$

- ▶ The vector $-\nabla L(\theta_k)$ is the direction of steepest local decrease in Euclidean norm.
- ▶ The scalar $\eta > 0$ is the learning rate. It turns a direction into a finite step.
- ▶ The update is local: it uses information at the current point, not a global view of the objective.

Gradient descent on a convex quadratic

- For

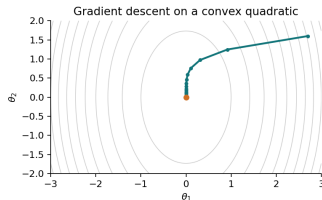
$$L(\theta) = \frac{1}{2} \theta^\top A \theta, \quad A = A^\top > 0,$$

the gradient is $\nabla L(\theta) = A\theta$.

- The update is

$$\theta_{k+1} = (I - \eta A) \theta_k.$$

- The plot shows the iterates moving across level sets toward the minimiser.



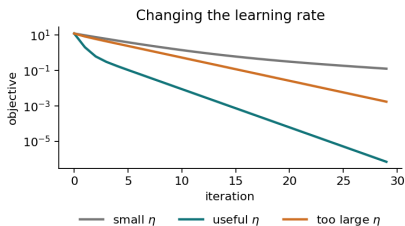
Learning rate: too small, useful, too large

- ▶ If η is too small, progress is stable but slow.
- ▶ If η is in a useful range, the loss decreases quickly.
- ▶ If η is too large, updates overshoot the minimiser and may diverge.
- ▶ For the quadratic $L(\theta) = \frac{1}{2}\theta^\top A\theta$, convergence requires

$$0 < \eta < \frac{2}{\lambda_{\max}(A)}.$$

Learning-rate curves

- ▶ A useful diagnostic is the objective value as a function of iteration.
- ▶ The same initial point and the same objective can behave very differently when only η changes.

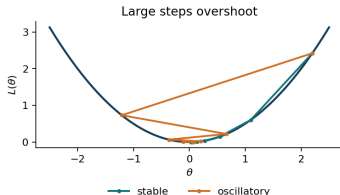


Overshooting in one dimension

- On $L(\theta) = \frac{1}{2}\theta^2$, the update is

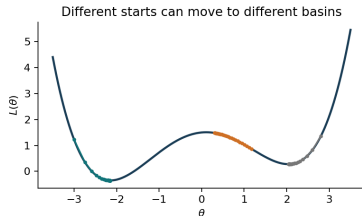
$$\theta_{k+1} = (1 - \eta)\theta_k.$$

- If $0 < \eta < 1$, the iterates approach the minimiser without changing sign.
- If $1 < \eta < 2$, the iterates oscillate while shrinking.
- If $\eta > 2$, the oscillations grow and the method diverges.



Non-convex objectives

- ▶ Neural-network losses are usually non-convex in θ .
- ▶ Different initialisations can move into different basins of attraction.
- ▶ A small gradient does not necessarily mean that the current point is the best possible predictor.



Local minima, saddle points and flat regions

- ▶ A point with $\nabla L(\theta) = 0$ is stationary. It may be a local minimum, a local maximum, or a saddle point.
- ▶ In high dimensions, saddle points and flat valleys are common; training can slow down even when no useful minimum has been reached.
- ▶ Permuting hidden units often leaves u_θ unchanged, so the same function may correspond to many parameter vectors.
- ▶ Repeat training with several seeds to measure sensitivity to initialisation.

Stochastic and mini-batch gradient descent

- Full-batch gradient descent uses all samples:

$$\nabla R_N(\theta) = \frac{1}{N} \sum_{i=1}^N \nabla_{\theta} \ell_i(\theta).$$

- Mini-batch SGD uses a subset B_k :

$$g_k = \frac{1}{|B_k|} \sum_{i \in B_k} \nabla_{\theta} \ell_i(\theta_k), \quad \theta_{k+1} = \theta_k - \eta g_k.$$

- The stochastic gradient is noisy but cheaper for large datasets. The noise can also help the iteration leave narrow regions.

Projected gradient descent

- If parameters must satisfy a constraint $\theta \in C$, projected GD uses

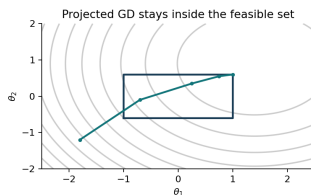
$$\theta_{k+1} = \Pi_C(\theta_k - \eta \nabla L(\theta_k)).$$

- The projection

$$\Pi_C(z) = \arg \min_{\theta \in C} \|\theta - z\|_2$$

returns the closest feasible point.

- This is useful for norm constraints, box constraints, and simple feasibility requirements.



Momentum, RMSProp and Adam

- Momentum smooths the update direction:

$$v_{k+1} = \mu v_k + g_k, \quad \theta_{k+1} = \theta_k - \eta v_{k+1}.$$

- RMSProp rescales coordinates using a running average of squared gradients.
- Adam combines momentum and adaptive coordinatewise scaling. It is not plain gradient descent.
- The optimiser, learning rate, batch size, dtype, seed, and update budget are part of the reported method.

Adam in PyTorch

- Create an Adam optimiser, then use it to update the parameters.

```
optimizer = torch.optim.Adam(  
    model.parameters(),  
    lr=1e-3,  
    weight_decay=0.0,  
)  
optimizer.zero_grad(set_to_none=True)  
loss.backward()  
optimizer.step()
```

- Report Adam and its settings when describing the training run.

Backpropagation is the chain rule

- For one neuron,

$$z = wx + b, \quad t = \tanh z,$$

$$\hat{y} = at + c, \quad L = \frac{1}{2}(\hat{y} - y)^2.$$

- With $r = \hat{y} - y$,

$$\frac{\partial L}{\partial a} = rt, \quad \frac{\partial L}{\partial c} = r, \quad \frac{\partial L}{\partial w} = ra(1 - t^2)x, \quad \frac{\partial L}{\partial b} = ra(1 - t^2).$$

- Backpropagation applies this chain rule efficiently across the full computational graph.

Reverse mode versus forward mode

- ▶ If $F : \mathbb{R}^p \rightarrow \mathbb{R}^q$, forward mode propagates products Jv and reverse mode propagates products $w^\top J$.
- ▶ Training has many parameters and one scalar loss: $p \gg q = 1$, so reverse mode is efficient.
- ▶ Coordinate derivatives of a scalar input are a different request. They differentiate with respect to x , not with respect to model parameters.
- ▶ Autograd is exact differentiation of executed operations up to floating-point rounding; it is not a finite-difference approximation.

The five-step training loop

- ▶ One full-batch update consists of clearing gradients, evaluating the model, forming the loss, backpropagating, and updating parameters.

```
model.train()
optimizer.zero_grad(set_to_none=True)
prediction = model(train_x)
loss = ((prediction - train_y)**2).mean()
loss.backward()
optimizer.step()
```

- ▶ Alongside this loop, record the loss and check predictions on validation data.

A safer training step

- Add checks that make common failures explicit.

```
prediction = model(train_x)
if prediction.shape != train_y.shape:
    raise ValueError("shape mismatch")
loss = ((prediction - train_y)**2).mean()
if loss.ndim != 0 or not torch.isfinite(loss):
    raise FloatingPointError("invalid loss")
optimizer.zero_grad(set_to_none=True)
loss.backward()
optimizer.step()
```

- These checks stop training when the shapes or loss are invalid.

Regularisation: changing the fitted solution

- ▶ Regularisation adds a preference among functions that fit the data similarly well.
- ▶ A common penalised objective is

$$L_{\text{reg}}(\theta) = R_N(\theta) + \lambda\Omega(\theta), \quad \lambda \geq 0.$$

- ▶ The penalty Ω encodes what is considered simple: small weights, sparse weights, smoothness, or another structural bias.
- ▶ Regularisation trades training fit for stability and generalisation.

L2 regularisation and weight decay

- L2 regularisation uses

$$\Omega(\theta) = \frac{1}{2} \|\theta\|_2^2, \quad L_{\text{reg}} = R_N(\theta) + \frac{\lambda}{2} \|\theta\|_2^2.$$

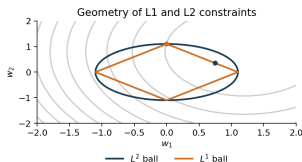
- Its gradient adds a shrinkage term:

$$\nabla L_{\text{reg}} = \nabla R_N(\theta) + \lambda \theta.$$

- For plain gradient descent,

$$\theta_{k+1} = (1 - \eta\lambda)\theta_k - \eta \nabla R_N(\theta_k).$$

- L2 tends to shrink weights smoothly rather than setting them exactly to zero.



L1 regularisation and sparsity

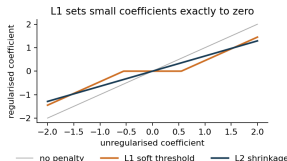
- L1 regularisation uses

$$\Omega(\theta) = \|\theta\|_1 = \sum_j |\theta_j|.$$

- The absolute-value penalty has corners. Those corners make exact zeros common.
- In the one-dimensional least-squares case, the solution is a soft-thresholding rule:

$$\hat{\theta}_{L1} = \text{sign}(z) \max\{|z| - \lambda, 0\}.$$

- This is why L1 is associated with feature selection and sparse coefficients.



Elastic net and practical penalties

- ▶ Elastic net combines L1 and L2:

$$L_{\text{reg}}(\theta) = R_N(\theta) + \lambda_1 \|\theta\|_1 + \frac{\lambda_2}{2} \|\theta\|_2^2.$$

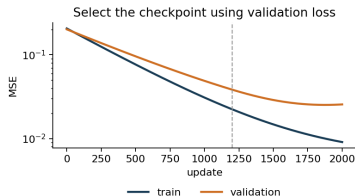
- ▶ L1 encourages sparsity; L2 stabilises and shrinks correlated parameters together.
- ▶ In neural networks, L2 weight decay is common; L1 can be used but may make optimisation less smooth.
- ▶ The penalty strength is a hyperparameter and must be chosen without looking at the test set.

Early stopping as implicit regularisation

- ▶ Early stopping does not add an explicit penalty to the objective.
- ▶ It restricts the optimisation path by selecting an earlier checkpoint:

$$\hat{k} = \arg \min_k R_{\text{val}}(\theta_k).$$

- ▶ In many problems, late iterations keep reducing training loss while validation loss stops improving.
- ▶ The checkpoint rule must be fixed before the test set is read.



Changing width

- ▶ Increasing width adds more hidden units per layer:

$$1 \rightarrow m \rightarrow m \rightarrow 1.$$

- ▶ The parameter count scales approximately like m^2 for two hidden layers because the middle matrix has size $m \times m$.
- ▶ Wider networks can fit more complicated functions, but they can also fit noise more easily.
- ▶ Width changes approximation, estimation, and optimisation at the same time; one plot rarely separates the three.

Changing depth

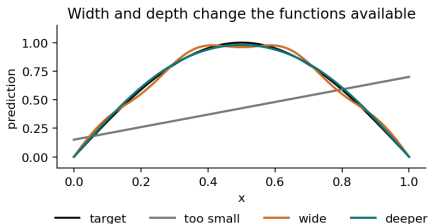
- ▶ Increasing depth adds more nonlinear compositions:

$$x \mapsto \sigma(W_L \sigma(\cdots \sigma(W_1 x + b_1) \cdots) + b_L).$$

- ▶ Depth can represent some compositional structures more efficiently than width alone.
- ▶ More depth also changes gradient flow: vanishing gradients, exploding gradients, and sensitivity to initialisation become more relevant.
- ▶ Residual connections and normalisation layers are common practical responses in larger models, but they are not needed for this small regression example.

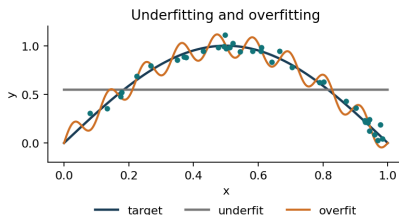
Width and depth in a regression plot

- ▶ The plot is schematic: changing architecture changes the set of functions the optimiser can select.
- ▶ Too little capacity underfits; enough capacity can track the signal; excessive flexibility can also track noise.



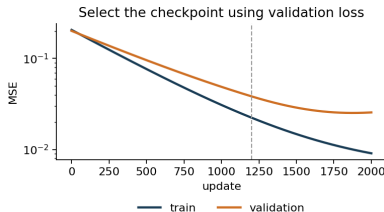
Underfitting and overfitting

- ▶ Underfitting: the model family is too limited or training has not reached a useful part of it.
- ▶ Overfitting: the model follows sample-specific fluctuations that do not generalise.
- ▶ Both can occur with neural networks; validation data are used to detect the trade-off.



Learning curves and validation

- ▶ Training loss measures fit to the data used for gradient updates.
- ▶ Validation loss estimates performance on data not used for updates.
- ▶ A widening gap between the two curves suggests overfitting or an overly long training run.



Checkpointing

- Save a copy of the best validation state, not a live reference to changing tensors.

```
model.eval()
with torch.no_grad():
    val_mse = ((model(val_x) - val_y)**2).mean().item()
if val_mse < best_val_mse:
    best_val_mse = val_mse
    best_state = copy.deepcopy(model.state_dict())
    best_step = step
```

- After training, reload the selected state and evaluate on the test split once.

Test evaluation and leakage

- ▶ Use the training split for gradient updates.
- ▶ Use the validation split for choices such as width, penalty strength, and selected checkpoint.
- ▶ Use the test split only after these choices are fixed.
- ▶ A number stops being a test estimate as soon as it influences a modelling decision.

Metrics: MSE, RMSE and MAE

- Mean squared error is

$$\text{MSE} = \frac{1}{M} \sum_{j=1}^M (\hat{y}_j - y_j)^2.$$

- Root mean squared error is

$$\text{RMSE} = \sqrt{\text{MSE}},$$

which has the same physical units as y .

- Mean absolute error is

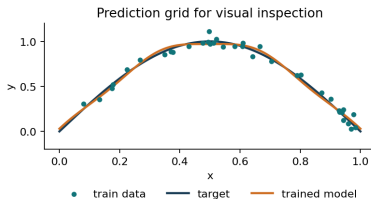
$$\text{MAE} = \frac{1}{M} \sum_{j=1}^M |\hat{y}_j - y_j|.$$

- Report which metric is being used; they answer related but different questions.

Plotting predictions on a grid

- A dense grid gives a visual check between observed points.

```
grid = torch.linspace(0, 1, 400,  
                      dtype=torch.  
                        float64)[: , None]  
model.eval()  
with torch.no_grad():  
    pred = model(grid).cpu().numpy  
    ()
```



Repeated seeds

- ▶ Randomness enters through data generation, data splits, initial weights, and sometimes mini-batch order.
- ▶ One seed gives one run; other initialisations can give different results.
- ▶ Repeating seeds helps distinguish systematic behaviour from a lucky or unlucky initialisation.
- ▶ Report mean, spread, and the exact protocol used to generate the runs.

Hyperparameter search without leakage

- ▶ Hyperparameters include width, depth, learning rate, batch size, penalty strength, optimiser, and training budget.
- ▶ Each candidate is trained on the training set and selected using validation performance.
- ▶ The test set is evaluated only for the selected protocol.
- ▶ If the test set is used to choose a hyperparameter, it becomes part of the selection process.

Common implementation bugs

- ▶ Shape mismatch: $(N, 1)$ compared with $(N,)$ can create an (N, N) residual matrix.
- ▶ Graph break: converting tensors to NumPy inside the loss prevents autograd from computing the required derivative.
- ▶ Gradient accumulation: forgetting `zero_grad` adds new gradients to old gradients.
- ▶ Test leakage: selecting the best epoch or architecture by test loss invalidates the final estimate.

Formula-to-code dictionary

- Model evaluation:

$$u_{\theta}(X) \longleftrightarrow \text{model}(X).$$

- Empirical risk:

$$\frac{1}{N} \sum_i (u_{\theta}(x_i) - y_i)^2 \longleftrightarrow ((\text{pred} - y)**2).mean().$$

- Parameter gradient:

$$\nabla_{\theta} L(\theta) \longleftrightarrow \text{loss.backward()}.$$

- Parameter update:

$$\theta \leftarrow \theta - \eta g \longleftrightarrow \text{optimizer.step()}.$$

A minimal experiment record

- ▶ Record the data protocol: target, noise level, sample sizes, and random seeds.
- ▶ Record the model: architecture, activation, parameter count, dtype, and initialisation policy.
- ▶ Record the training protocol: optimiser, learning rate, batch size, penalty, update budget, and checkpoint rule.
- ▶ Record the evaluation protocol: metric, split, and whether the test result was used only once.

What to remember

- ▶ Supervised regression fits a predictor by minimising a loss on observed data.
- ▶ Gradient descent follows local gradient information; the learning rate, stochasticity, constraints, and optimiser variant change the path.
- ▶ Regularisation changes which fitted functions are preferred: L2 shrinks, L1 promotes sparsity, and early stopping selects along the training path.
- ▶ Choose width and depth using validation data. Report the data, training settings, and test error so others can assess the result.